

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ**

**ОТЧЕТ
по лабораторной работе №4
по дисциплине «Вычислительная математика»**

Студент гр. 4341

Баженов К.Э.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург
2026

Цель работы.

Изучить метод хорд для численного решения нелинейных уравнений, реализовать программу нахождения корня уравнения с заданной точностью, исследовать скорость сходимости метода и оценить чувствительность метода к ошибкам в исходных данных.

Задание.

1. Отделить корень заданного нелинейного уравнения.
2. Реализовать метод хорд для нахождения корня с заданной точностью.
3. Провести вычисления при различных значениях ϵ_{ps} в диапазоне от 0.1 до 0.000001.
4. Исследовать скорость сходимости метода теоретически и экспериментально.
5. Исследовать чувствительность метода к ошибкам исходных данных, моделируя их с помощью округления.

Вариант 1: $f(x) = x^2 - 5.0 * \sin(x)$

Отделение корней уравнения.

Рассмотрим функцию $f(x) = x^2 - 5\sin(x)$. Она определена и непрерывна на всей числовой оси.

Её производная равна:

$$f'(x) = 2x - 5\cos(x)$$

Аналитически получить все корни уравнения $x^2 - 5\sin(x) = 0$ в явном виде невозможно, поэтому исследуем функцию и область возможного расположения корней.

Из уравнения $x^2 = 5\sin(x)$ следует, что правая часть должна быть неотрицательной, то есть $\sin(x) \geq 0$. Кроме того, так как $\sin(x) \leq 1$, то $x^2 = 5\sin(x) \leq 5$, следовательно, $|x| \leq \sqrt{5}$.

Значит, все корни могут находиться только на отрезке $[-\sqrt{5}; \sqrt{5}]$.

Так как при $x < 0$ в рассматриваемой области $\sin(x) < 0$, равенство $x^2 = 5\sin(x)$ невозможно, поэтому корни следует искать только при $x \geq 0$, то есть на отрезке $[0; \sqrt{5}]$.

Очевидно, что при $x = 0$:

$$f(0) = 0^2 - 5\sin(0) = 0, \text{ следовательно, } x = 0 \text{ является корнем уравнения.}$$

Локализуем второй корень.

При $x = 2$:

$$f(2) = 2^2 - 5\sin(2) = 4 - 5\sin(2) \approx 4 - 4.546 = -0.546 < 0$$

При $x = \sqrt{5}$:

$$f(\sqrt{5}) = (\sqrt{5})^2 - 5\sin(\sqrt{5}) = 5 - 5\sin(\sqrt{5}) > 0, \text{ так как } \sin(\sqrt{5}) < 1.$$

Следовательно, на отрезке $[2; \sqrt{5}]$ функция меняет знак, а значит, по теореме Больцано–Коши на этом отрезке существует корень.

Для метода хорд будет взят отрезок $[2; 2.5]$.

Приведённые рассуждения подтверждаются графически (см. рис. 1).

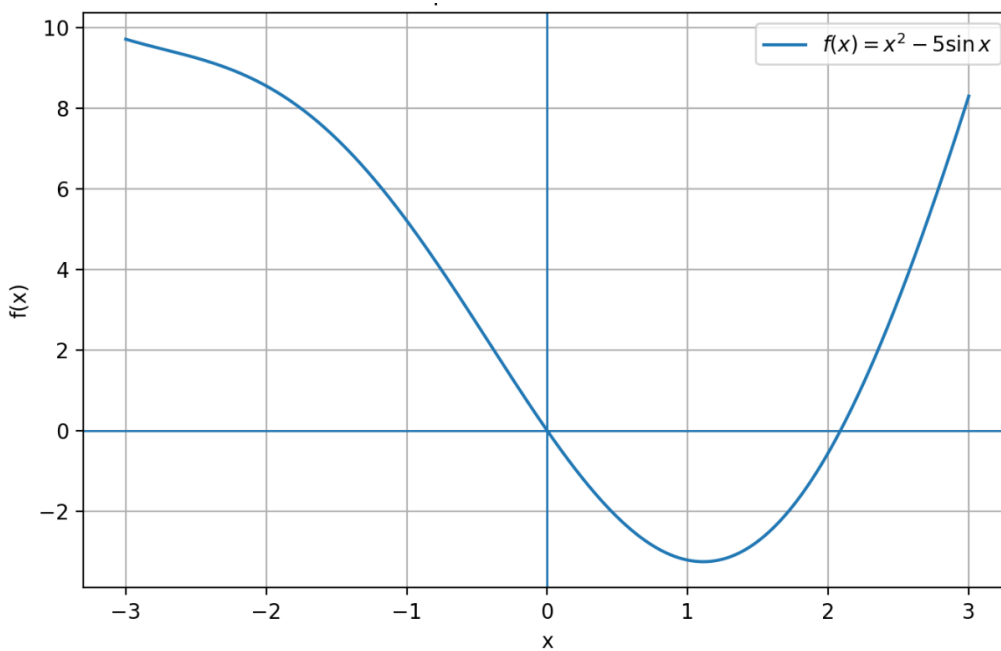


Рис. 1 – График $f(x)$

Разработка программы.

Программа, реализующая метод хорд, была написана на языке программирования C++.

Была написана функция `double F(double x)` для вычисления значения функции $f(x)$:

```
double F(double x) {  
    return x * x - 5.0 * sin(x);  
}
```

Функция `double horda(double left, double right, double eps, double delta, int &N)` реализует метод хорд. На каждом шаге строится хорда через точки графика функции на концах текущего интервала, после чего определяется точка её

пересечения с осью абсцисс. Эта точка принимается как новое приближение к корню.

Аргументы:

left, right – границы отрезка, на котором локализован корень;

eps – точность вычисления корня;

delta – точность округления функции для симуляции погрешности;

N – переменная, в которую записывается количество выполненных итераций.

Также была реализована функция *double Round(double x, double delta)*, возвращающая значение *x*, округлённое с точностью *delta*.

Аргументы:

x – значение для округления;

delta – точность округления.

Для исследования чувствительности метода к ошибкам в исходных данных в программе выполнялось округление значений функции перед их использованием в вычислениях.

Разработанный программный код приведён в приложении А.

Исследование скорости сходимости.

Скорость сходимости метода хорд является линейной, то есть погрешность на каждом шаге уменьшается примерно в одно и то же число раз: $|x_{n+1} - c| \approx q \cdot |x_n - c|$, где $0 < q < 1$, *c* — точный корень. Для подсчёта числа итераций в функции *horda* использовался счётчик количества итераций. При каждой итерации значение счётчика увеличивалось на 1.

Ниже приведён график теоретической и практической зависимости числа итераций от ϵ (см. рис. 2).

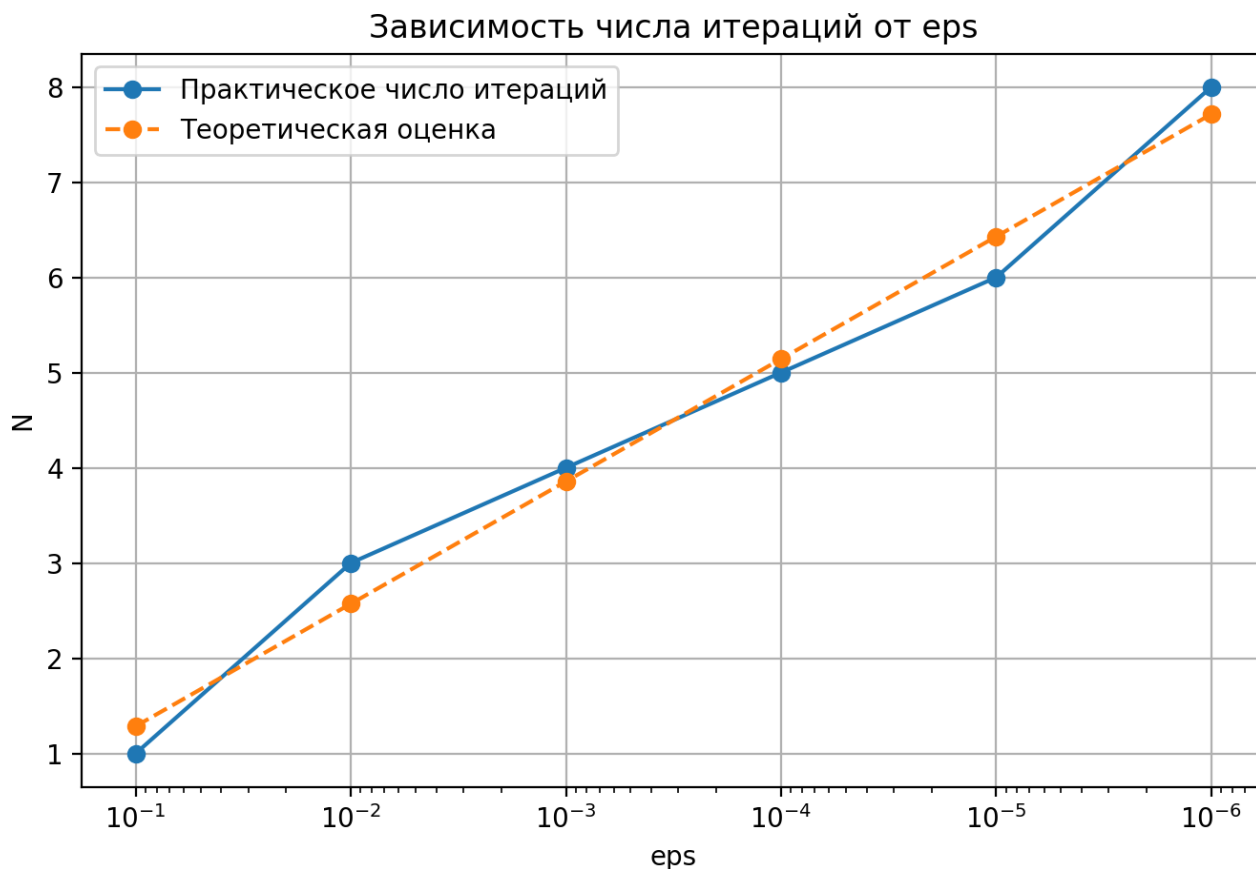


Рис. 2 – Зависимость числа итераций от ϵ

Из графика видно, что при уменьшении значения ϵ число итераций возрастает. Метод хорд показывает достаточно быструю сходимость и позволяет получить значение корня за меньшее число шагов по сравнению с методом бисекции.

Тестирование.

Результаты работы программы для различных значений ϵ приведены в таблице 1.

Таблица 1 – Результаты тестирования

Точность вычислений ϵ	Результат	Число итераций
0.1	2.071828	1
0.01	2.085585	3

0.001	2.085880	4
0.0001	2.085926	5
0.00001	2.085933	6
0.000001	2.085935	8

Чувствительность метода к ошибкам в исходных данных.

Для исследования чувствительности метода хорд к ошибкам в исходных данных в программу была добавлена функция $\text{Round}(x, \text{delta})$, выполняющая округление значений функции $F(x)$ с точностью delta .

Далее для каждого значения eps метод хорд запускался при разных значениях delta , чтобы исследовать чувствительность метода к исходным данным. Результаты вычислений представлены на графике (рис. 3).

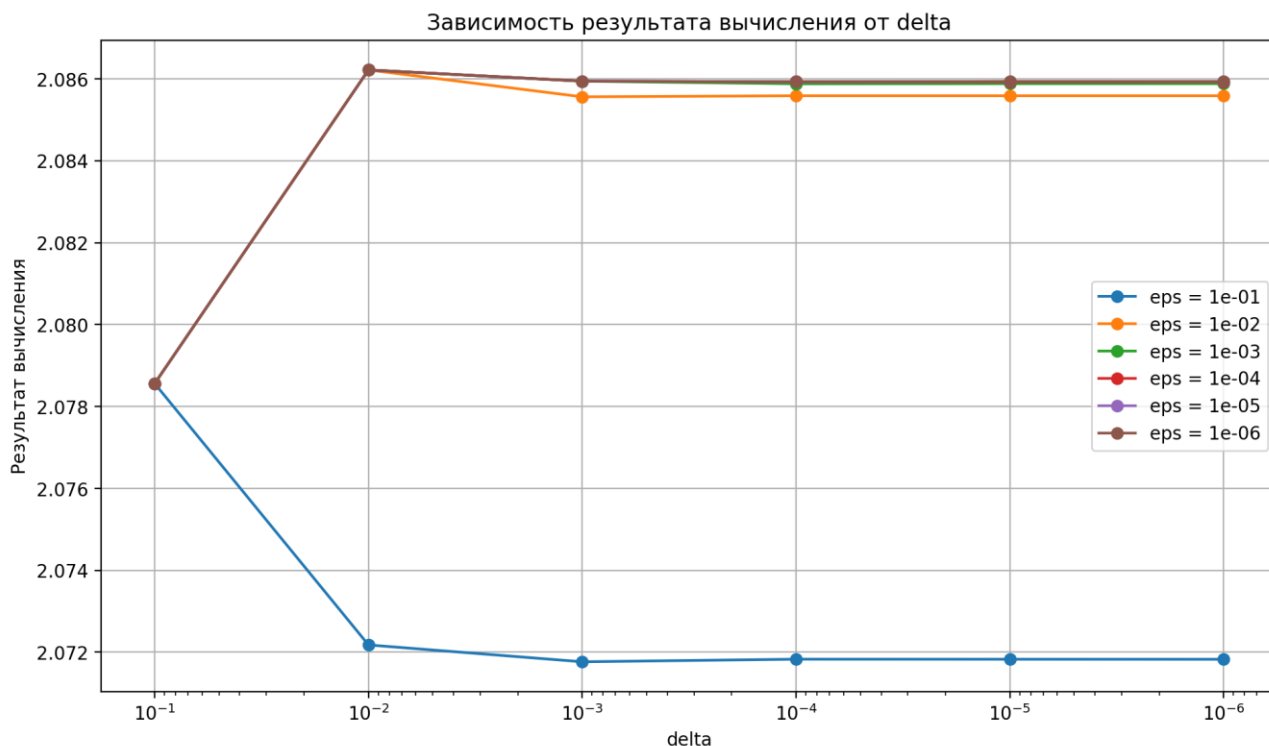


Рис. 3 – Зависимость результата вычисления от delta

Результат вычислений изменяется незначительно при малых значениях delta , что свидетельствует о достаточной устойчивости метода хорд к умеренным

ошибкам округления. Однако при более грубом округлении влияние погрешностей может возрасти сильнее, чем в методе бисекции, поскольку метод хорд использует значения функции непосредственно в формуле вычисления следующего приближения.

Вывод.

В ходе работы был изучен и реализован метод хорд для численного решения нелинейного уравнения с заданной точностью. Была разработана программа на языке C++, позволяющая находить корень уравнения и исследовать поведение метода при различных значениях точности. В процессе выполнения работы был отделён корень уравнения, проведены вычисления при различных значениях ϵ_{ps} , исследована скорость сходимости метода и выполнен анализ чувствительности к ошибкам в исходных данных.

ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>
#include <cstdlib>

using namespace std;

double Round(double x, double delta) {
    if (delta <= 1E-9) {
        cout << "Неверное задание точности округления\n";
        exit(1);
    }

    if (x > 0.0)
        return delta * (long) ((x / delta) + 0.5);
    else
        return delta * (long) ((x / delta) - 0.5);
}

double F(double x) {
    return x * x - 5.0 * sin(x);
}

double F_rounded(double x, double delta) {
    if (delta == 0) return F(x);
    return Round(F(x), delta);
}

double horda(double left, double right, double eps, double
delta, int &N) {
    double fLeft = F_rounded(left, delta);
    double fRight = F_rounded(right, delta);
    double X, Y;

    if (fLeft * fRight > 0.0) {
        cout << "Неверное задание интервала\n";
        exit(1);
    }
    if (eps <= 0.0) {
        cout << "Неверное задание точности\n";
        exit(1);
    }
```

```

    }

    N = 0;
    if (fLeft == 0.0) return left;
    if (fRight == 0.0) return right;

    do {
        X = left - (right - left) * fLeft / (fRight - fLeft);
        Y = F_rounded(X, delta);
        if (Y == 0.0) return (X);
        if (Y * fLeft < 0.0) {
            right = X;
            fRight = Y;
        }
        else {
            left = X;
            fLeft = Y;
        }
        N++;
    } while (fabs(Y) >= eps);

    return (X);
}

int main() {
    ofstream out("output.txt");
    if (!out) {
        cout << "Не удалось открыть файл output.txt\n";
        return 1;
    }

    double left = 2.0;
    double right = 2.5;

    out << setprecision(6);

    double eps_values[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6};
    double delta_values[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 0};

    for (double eps : eps_values) {
        int N;

        for (double delta : delta_values) {
            double root = horda(left, right, eps, delta, N);
            out << fixed << "Eps: " << eps
                << " Delta: " << delta
                << " N: " << N
                << " Root: " << root << '\n';
        }
    }
}

```

```
        }  
    }  
  
    out.close();  
    return 0;  
}
```